*Технология разработки программного обеспечения*

# Практическая работа №3
# Космический тир

*Реализация паттернов из лекции 8*

Выполнил(а): _____ Группа: _____

Преподаватель: _____

### Цель работы

Разработать игру на Python с игровым циклом и методом update; сделать движение независимым от FPS, переиспользовать пули и частицы, выделить поведение врагов в состояния и сократить количество проверок столкновений пространственной сеткой.

### Объём выполненного задания

Выполнены этапы 0–5, а также все четыре задания со звёздочкой: очередь событий, турели из компонентов, временный бонус тройного выстрела и жизни игрока с экраном окончания игры.

Предоставлены два PDF. Исходная заготовка не была приложена, поэтому создан самостоятельный проект по требованиям практики. Сохранены названия пяти упомянутых пользователем модулей; точное совпадение с отсутствовавшей заготовкой не проверялось.

### Средства разработки и структура

Требования: Python 3.10+, pygame-ce 2.5.x. Проверка выполнена на Python 3.12.14 и pygame-ce 2.5.8. Окно 800×600. Рисование средствами pygame, русский шрифт приложен с лицензией.

| Файл | Назначение |
|---|---|
| main.py | Ввод → обновление → отрисовка, измерение dt |
| game.py | Мир, столкновения, HUD, подписчики событий |
| entities.py | Корабль, пули, частицы, бонусы |
| enemy.py | Враги и Patrol, Chase, Attack, Flee |
| pool.py / grid.py | Универсальный пул / пространственная сетка |
| components.py | GameObject, Health, Shooter, Turret |
| event_queue.py / config.py | Очередь событий / размеры, цвета, шрифты |

# 1. Delta time, пул пуль и состояния

### Этап 0. Запуск и игровой цикл

Цикл считывает события и клавиши, обновляет мир, затем рисует кадр. Начальная сцена содержит корабль, звёздное небо, 8 обычных врагов и 2 турели. Для запуска устанавливается pygame-ce и выполняется python main.py.

### Этап 1. Независимость от частоты кадров

```
dt = min(clock.tick(game.fps_limit) / 1000.0, .05)
game.update(dt, pygame.key.get_pressed())
game.draw(screen, clock.get_fps())
```

Позиция изменяется на скорость × dt. Скорости: корабль — 300 px/s, пуля — 600 px/s, патруль — 120 px/s, погоня вниз — 60 px/s, атака — 400 px/s. Перезарядка — 0.15 s. При удержании пробела сохраняется дробный остаток таймера; простой не накапливает будущие выстрелы. L переключает ограничение 60/20 FPS.

### Этап 2. Пул пуль

В Game используется Pool(Bullet, 20). get() активирует свободную пулю, launch() задаёт её координаты и скорость. При попадании или вылете пуля выключается. active() возвращает итератор активных объектов. Если все объекты заняты, пул расширяется до необходимого размера; конструктор не вызывается при обычном повторном использовании.

HUD показывает активные и созданные объекты. За 30 s непрерывной одиночной стрельбы выполняются 200 выстрелов, при этом размер пула остаётся 20. Пули турелей хранятся в отдельном пуле.

### Этап 3. Паттерн «Состояние»

Enemy хранит state и делегирует обновление state.update(enemy, player, dt). Метод состояния возвращает следующий объект состояния либо None. Имя и цвет принадлежат классу состояния; F1 выводит их над врагами.

| Состояние | Поведение и переход |
|---|---|
| Patrol — бирюзовый | Движение по горизонтали; дистанция по X < 100 px → Chase. |
| Chase — оранжевый | Приближение к игроку; y > 300 → Attack; дистанция по X > 200 → Patrol. |
| Attack — красный | Пикирование 400 px/s. За нижним краем — возрождение и Patrol. |
| Flee — фиолетовый | Попадание оставило 1 HP: полёт вверх 180 px/s в течение 2 s, затем возрождение и Patrol. |

Обычный враг имеет 3 HP. Возрождение после бегства восстанавливает здоровье и возвращает врага на поле; это выбранное правило для предотвращения патруля вне экрана. За бегство и проход за экран очки не начисляются.

# 2. Сетка, частицы и события

### Этап 4. Пространственная сетка

Grid имеет размер клетки 48 px и методы clear(), add(obj), near(x, y). Перед проверкой враги распределяются по клеткам. near() обходит свою клетку и восемь соседних. При F1 видны линии сетки; G переключает кандидатов сетки и полный список врагов.

Точная проверка использует относительный отрезок движения пули и врага за кадр. Это позволяет обнаружить встречное пересечение даже тогда, когда в конце кадра круги уже разошлись. Для сетки берутся окрестности обоих концов пути пули. Порядок кандидатов соответствует полному перебору, чтобы режимы одинаково обрабатывали перекрывающихся врагов.

Обрабатывается и последний отрезок пули, которая вылетела за экран в текущем кадре. Враг, уничтоженный и возрождённый в этом кадре, исключается из оставшихся проверок, чтобы одна смерть не учитывалась повторно.

| Одинаковая контрольная сцена | Перебор | Сетка |
|---|---|---|
| 158 врагов, 40 пуль: точные проверки | 3160 | 221 |
| Результаты попаданий | Совпадают | Совпадают |

В этой сцене число проверок снижено в 14.3 раза. 40 пуль специально активированы стендом для нагрузки. Это не режим обычной стрельбы, в котором достаточно начальных 20 объектов. Исходные результаты сохранены в docs/benchmark.json.

### Этап 5. Частицы на пуле

Для Particle используется тот же класс Pool: Pool(Particle, 300). При каждом попадании публикуется эффект 15 искр цвета текущего состояния врага. При уничтожении дополнительно создаются 40 золотых искр: суммарно 55 на смертельное попадание.

У частицы есть позиция, скорость, цвет, время жизни и active. Направление случайное, скорость 50–250 px/s, жизнь 0.3–0.8 s. По истечении времени объект выключается и позже используется повторно. При большом одновременном числе эффектов пул может расшириться; после завершения эффектов объекты остаются доступными для повторного использования.

### Задание ★. Очередь событий

on_hit() отвечает за урон, публикацию enemy_hit и, при смерти, enemy_killed. Счёт, искры и выпадение бонуса — независимые подписчики. EventQueue.process() вызывается после столкновений, поэтому логика эффектов не вмешивается в обход объектов.

```
self.events.subscribe('enemy_killed', self._add_score)
self.events.subscribe('enemy_killed', self._death_sparks)
self.events.subscribe('enemy_killed', self._drop_bonus)
```

HitEvent — неизменяемый снимок координат и цвета. Он сохраняет место попадания даже после немедленного возрождения врага. События, опубликованные обработчиком события, ждут следующего process().

# 3. Компоненты, бонусы и жизни

### Задание ★. Компонентная турель

GameObject получает список независимых компонентов и вызывает update(obj, dt) каждого. Турель собрана из Health(3) и Shooter(1.2). Движущего компонента у неё нет, поэтому она остаётся на месте. Компонент Shooter каждые 1.2 s вызывает функцию выстрела вниз. Пули турели летят 250 px/s и используют отдельный Pool(Bullet, 20).

Такой состав позволяет менять стрельбу и здоровье отдельно, не создавая ветви наследования для каждой комбинации возможностей. Турели являются целями для пуль игрока и после уничтожения возрождаются.

### Задание ★. Временный бонус

При уничтожении врага или турели с вероятностью 25% выпадает квадрат с цифрой 3. Он падает со скоростью 90 px/s. Подбор включает три пули с разными горизонтальными скоростями на 5 s. Повторный подбор обновляет таймер до 5 s; затем стрельба автоматически возвращается к одиночной. Остаток времени показан в HUD.

### Задание ★. Жизни игрока

У игрока 3 жизни. Пуля турели и столкновение с атакующим врагом отнимают одну жизнь. После урона действует защита 1.5 s и мигание. Она предотвращает потерю нескольких жизней в одном кадре. Столкнувшийся враг возрождается. При нуле жизней обновление мира прекращается и появляется экран «Игра окончена». R создаёт новый игровой мир с нулевым счётом и тремя жизнями.

### Управление и ручная приёмка

| Клавиша | Действие / что проверить |
|---|---|
| ← / →, Пробел | Движение и стрельба; зажать пробел на 30 s, наблюдать размер пула. |
| L | 60 ↔ 20 FPS. Скорость движения должна сохраниться. |
| F1 | Сетка и подписи состояний, цвет соответствует подписи. |
| = / + | Добавить 50 врагов. Три нажатия: 158 обычных врагов и 2 турели. |
| G | Сетка ↔ перебор. Сравнить счётчик проверок во время стрельбы. |
| R / Esc | Новая игра после окончания / выход. |

Дополнительно: дважды попасть в обычного врага и проверить Flee; уничтожить врага и увидеть взрыв; подобрать бонус и дождаться истечения пяти секунд; проверить потерю жизни, окончание игры и перезапуск.

### Запуск и автоматическая проверка

```
python -m pip install -r requirements.txt
python main.py
python -m unittest discover -v
python benchmark.py
```

В README.txt также есть команды для Windows, Linux/macOS и отдельной виртуальной среды. Скрипты start.bat и start.sh поддерживают локальную .venv.

# 4. Иллюстрация готовой игры

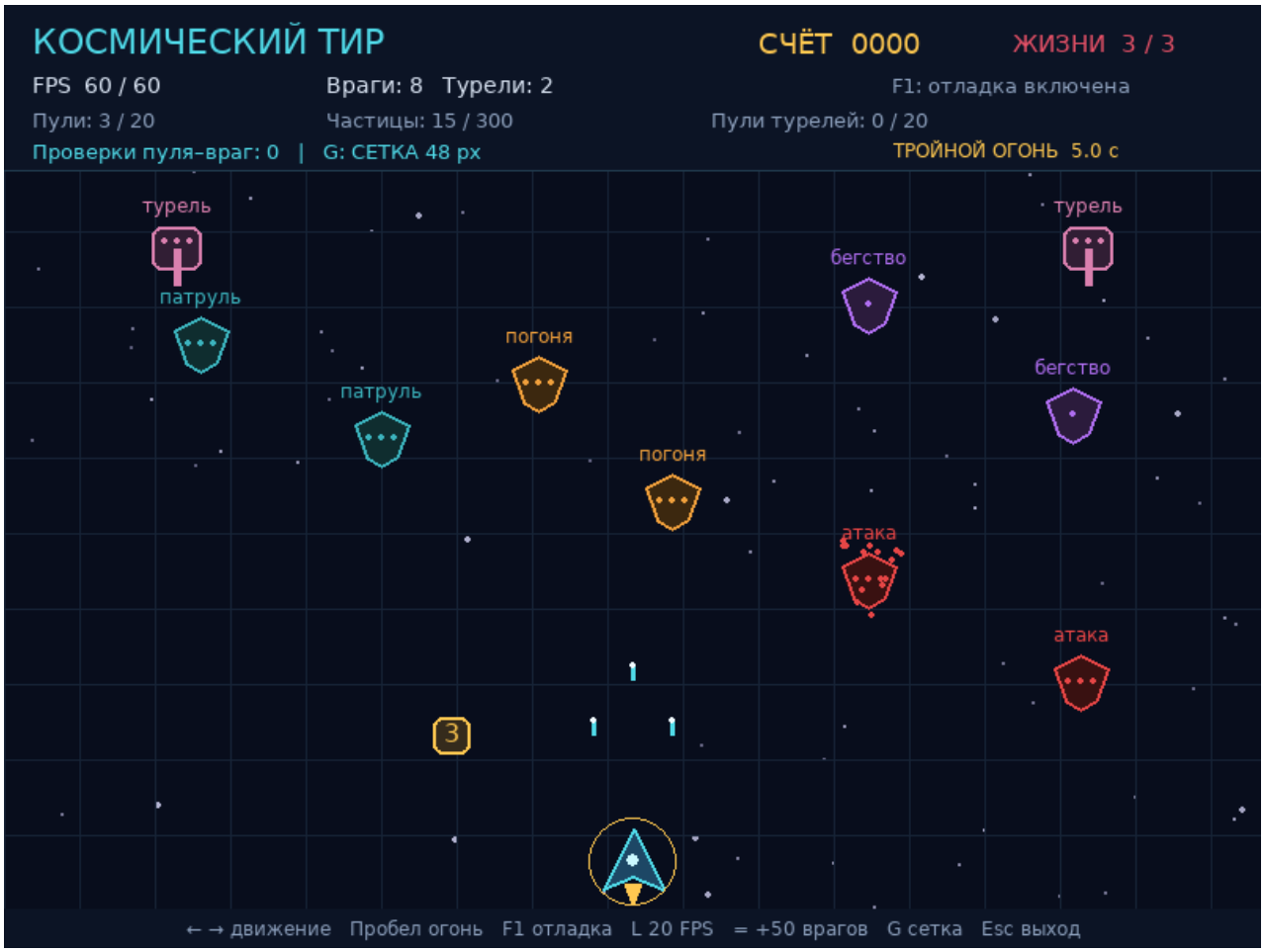



**Рисунок 1. Демонстрационный кадр: сетка 48 px, все четыре состояния, две турели, пули, частицы и бонус. Враги заранее размещены для наглядности; изображение иллюстрирует функции игры и не является нагрузочным измерением.**

### Обозначения на экране

Бирюзовый — патруль; оранжевый — погоня; красный — атака; фиолетовый — бегство. Розовые прямоугольники — турели. Светлые точки внутри врагов показывают оставшееся здоровье. Жёлтый квадрат «3» — временный бонус.

Сверху расположены счёт, жизни, FPS и его ограничение, количество врагов, размеры пулов и число проверок пуля–враг. В формате «3 / 20» первое число — активные объекты, второе — общее число созданных. Снизу показаны клавиши управления.

Полноразмерные изображения: docs/gameplay.png — кадр после программно поданных команд управления; docs/debug.png — иллюстрация состояний; docs/game_over.png — экран окончания игры.

# 5. Результаты проверки

Команда python -m unittest discover -v: 35 тестов, ошибок и провалов нет. Протокол — docs/tests.txt. Проверки выполнялись в безоконной среде; главный цикл дополнительно запускался через SDL dummy и сохранял кадры PNG.

| Проверка | Полученный результат |
|---|---|
| Движение 1 s при 20 и 60 FPS | 300 px в обоих случаях |
| Стрельба 30 s при 20 и 60 FPS | 200 выстрелов, размер пула 20 |
| Стрельба при кадрах 16–17 ms | 200 выстрелов за 30 s, без потери остатка таймера |
| 100 разнесённых во времени взрывов | Размер пула частиц остаётся 300 |
| Сетка и перебор на одинаковых данных | Одинаковые попадания, 3160 → 221 проверка |
| События после возрождения | Искры остаются в координатах попадания |
| Последний отрезок пули у края | Попадание учитывается до окончательного удаления из кадра |
| Бонус / урон / окончание / R | Таймер 5 s; защита от повторного урона; мир заморожен; перезапуск |

### Нагрузочный стенд

python benchmark.py выполняет 300 обновлений и отрисовок в pygame.Surface для каждой сцены. Во время стенда игрок защищён от урона, чтобы окончание игры не остановило нагрузку; это условие действует только в benchmark.py.

| Обычные враги + турели | Среднее update+draw | Пулы пуль / частиц |
|---|---|---|
| 158 + 2 | 1.832 ms | 20 / 300 |
| 508 + 2 | 3.582 ms | 20 / 300 |

Эти числа характеризуют только время вычислений и рисования в памяти на данной машине. В них нет оконного вывода и синхронизации дисплея. Это не замер реального FPS и не гарантия одинаковой производительности на другом компьютере.

### Границы проверки

Оригинальные пять исходных файлов не были предоставлены: проверялось выполнение требований PDF, а не совместимость с их неизвестным API. В среде выполнения нет интерактивного дисплея; отрисовка проверена по PNG, команды управления — тестами и программной подачей ввода. Ручную приёмку на компьютере пользователя можно выполнить по разделу 3 и README.txt.

### Использованные материалы

1. «8. Паттерны игрового программирования.pdf»: игровой цикл, update, Pool, State, Component, Grid, Event Queue.
2. «8. Практика.pdf»: «Космический тир: паттерны в игре», этапы 0–5 и задания со звёздочкой.
3. pygame-ce 2.5.8 и стандартная библиотека Python: unittest, dataclasses, random, math.